

Bit

位运算

位运算就是基于整数的二进制表示进行的运算。由于计算机内部就是以二进制来存储数据，位运算是相当快的。

基本的位运算共 种，分别为按位与、按位或、按位异或、按位取反、左移和右移。

为了方便叙述，下文中省略「按位」。

与、或、异或

这三者都是两数间的运算，因此在这里一起讲解。

它们都是将两个整数作为二进制数，对二进制表示中的每一位逐一运算。

运算	运算符	数学符号表示	解释
与	<code>&</code>	、	只有两个对应位都为 1 时才为 1
或	<code> </code>	、	只要两个对应位中有一个 1 时就为 1
异或	<code>^</code>	、	只有两个对应位不同时才为 1

注意区分逻辑与（对应的数学符号为 $\wedge$ ）和按位与、逻辑或（ $\vee$ ）和按位或的区别。网络中的资料中使用的符号多有不规范之处，以上下文为准。

异或运算的逆运算是它本身，也就是说两次异或同一个数最后结果不变，即 $a \oplus b \oplus b = a$ 。

举例：

取反

取反是对一个数 进行的位运算，即单目运算。

取反暂无默认的数学符号表示，其对应的运算符为 `~`。它的作用是把 的二进制补码中的 0 和 1 全部取反（0 变为 1，1 变为 0）。有符号整数的符号位在 `~` 运算中同样会取反。

补码：在二进制表示下，正数和 0 的补码为其本身，负数的补码是将其对应正数按位取反后加一。

举例（有符号整数）：

的补码

左移和右移

`num << i` 表示将 的二进制表示向左移动 `i` 位所得的值。

`num >> i` 表示将 的二进制表示向右移动 `i` 位所得的值。

举例：

移位运算中如果出现如下情况，则其行为未定义：

1. 右操作数（即移位数）为负值；
2. 右操作数大于等于左操作数的位数；

例如，对于 `int` 类型的变量 `a`，`a<<-1` 和 `a<<32` 都是未定义的。

对于左移操作，需要确保移位后的结果能被原数的类型容纳，否则行为也是未定义的。¹ 对一个负数执行左移操作也未定义。²

对于右移操作，右侧多余的位将会被舍弃，而左侧较为复杂：对于无符号数，会在左侧补 0；而对于有符号数，则会用最高位的数（其实就是符号位，非负数为 0，负数为 1）补齐。³

复合赋值位运算符

和 `+=`，`-=` 等运算符类似，位运算也有复合赋值运算符：`&=`，`|=`，`^=`，`<<=`，`>>=`。（取反是单目运算，所以没有。）

关于优先级

位运算的优先级低于算术运算符（除了取反），而按位与、按位或及异或低于比较运算符（详见 [C++ 运算符优先级总表](#)），所以使用时需多加注意，在必要时添加括号。

位运算的应用

位运算一般有三种作用：

1. 高效地进行某些运算，代替其它低效的方式。
2. 表示集合（常用于 [状压 DP](#)）。
3. 题目本来就要求进行位运算。

需要注意的是，用位运算代替其它运算方式（即第一种应用）在很多时候并不能带来太大的优化，反而会使代码变得复杂，使用时需要斟酌。（但像「乘 2 的非负整数次幂」和「除以 2 的非负整数次幂」就最好使用位运算，因为此时使用位运算可以优化复杂度。）

有关 2 的幂的应用

由于位运算针对的是变量的二进制位，因此可以推广出许多与 2 的整数次幂有关的应用。

将一个数乘（除）2 的非负整数次幂：



C++Python

```
1 int mulPowerOfTwo(int n, int m) { // 计算 n*(2^m)
2   return n << m;
3 }
4 int divPowerOfTwo(int n, int m) { // 计算 n/(2^m)
5   return n >> m;
6 }
```

```
1 def mulPowerOfTwo(n, m): # 计算 n*(2^m)
2   return n << m
3 def divPowerOfTwo(n, m): # 计算 n/(2^m)
4   return n >> m
```

Warning

我们平常写的除法是向 取整，而这里的右移是向下取整（注意这里的区别），即当数大于等于 时两种方法等价，当数小于 时会有区别，如： $-1 / 2$ 的值为 ，而 $-1 \gg 1$ 的值为 。

取绝对值

在某些机器上，效率比 $n > 0 ? n : -n$ 高。



C++Python

```
1 int Abs(int n) {
2     return (n ^ (n >> 31)) - (n >> 31);
3     /* n>>31 取得 n 的符号，若 n 为正数，n>>31 等于 0，若 n 为负数，n>>31 等于 -1
4     若 n 为正数 n^0=n，数不变，若 n 为负数有 n^(-1)
5     需要计算 n 和 -1 的补码，然后进行异或运算，
6     结果 n 变号并且为 n 的绝对值减 1，再减去 -1 就是绝对值 */
7 }
```

```
1 def Abs(n):
2     return (n ^ (n >> 31)) - (n >> 31)
3     """
4     n>>31 取得 n 的符号，若 n 为正数，n>>31 等于 0，若 n 为负数，n>>31 等于 -1
5     若 n 为正数 n^0=n，数不变，若 n 为负数有 n^(-1)
6     需要计算 n 和 -1 的补码，然后进行异或运算，
7     结果 n 变号并且为 n 的绝对值减 1，再减去 -1 就是绝对值
8     """
```

取两个数的最大/最小值

在某些机器上，效率比 $a > b ? a : b$ 高。



C++Python

```
1 // 如果 a >= b, (a - b) >> 31 为 0，否则为 -1
2 int max(int a, int b) { return (b & ((a - b) >> 31)) | (a & (~(a - b) >> 31)); }
3 int min(int a, int b) { return (a & ((a - b) >> 31)) | (b & (~(a - b) >> 31)); }
```

```
1 # 如果 a >= b, (a - b) >> 31 为 0，否则为 -1
2 def max(a, b):
3     return b & ((a - b) >> 31) | a & (~(a - b) >> 31)
4 def min(a, b):
5     return a & ((a - b) >> 31) | b & (~(a - b) >> 31)
```

判断两非零数符号是否相同



C++Python

```
1 bool isSameSign(int x, int y) { // 有 0 的情况例外
2     return (x ^ y) >= 0;
3 }
```

```
1 # 有 0 的情况例外
2 def isSameSign(x, y):
3     return (x ^ y) >= 0
```

交换两个数

▼ 该方法具有局限性

这种方式只能用来交换两个整数，使用范围有限。

对于一般情况下的交换操作，推荐直接调用 `algorithm` 库中的 `std::swap` 函数。

```
1 void swap(int &a, int &b) { a ^= b ^= a ^= b; }
```

操作一个数的二进制位

获取一个数二进制的某一位：



C++Python

```
1 // 获取 a 的第 b 位，最低位编号为 0
2 int getBit(int a, int b) { return (a >> b) & 1; }
```

```
1 # 获取 a 的第 b 位，最低位编号为 0
2 def getBit(a, b):
3     return (a >> b) & 1
```

将一个数二进制的某一位设置为：



C++Python

```
1 // 将 a 的第 b 位设置为 0，最低位编号为 0
2 int unsetBit(int a, int b) { return a & ~(1 << b); }
```

```
1 # 将 a 的第 b 位设置为 0，最低位编号为 0
2 def unsetBit(a, b):
3     return a & ~(1 << b)
```

将一个数二进制的某一位设置为：



C++Python

```
1 // 将 a 的第 b 位设置为 1，最低位编号为 0
2 int setBit(int a, int b) { return a | (1 << b); }
```

```
1 # 将 a 的第 b 位设置为 1，最低位编号为 0
2 def setBit(a, b):
3     return a | (1 << b)
```

将一个数二进制的某一位取反：



C++Python

```
1 // 将 a 的第 b 位取反，最低位编号为 0
2 int flapBit(int a, int b) { return a ^ (1 << b); }
```

```

1 # 将 a 的第 b 位取反，最低位编号为 0
2 def flipBit(a, b):
3     return a ^ (1 << b)

```

这些操作相当于将一个 位整型变量当作一个长度为 的布尔数组。

汉明权重

汉明权重是一串符号中不同于（定义在其所使用的字符集上的）零符号（zero-symbol）的个数。对于一个二进制数，它的汉明权重就等于它 的个数（即 `popcount`）。

求一个数的汉明权重可以循环求解：我们不断地去掉这个数在二进制下的最后一位（即右移 位），维护一个答案变量，在除的过程中根据最低位是否为 更新答案。

代码如下：

```

1 // 求 x 的汉明权重
2 int popcount(int x) {
3     int cnt = 0;
4     while (x) {
5         cnt += x & 1;
6         x >>= 1;
7     }
8     return cnt;
9 }

```

求一个数的汉明权重还可以使用 `lowbit` 操作：我们将这个数不断地减去它的 `lowbit`⁴，直到这个数变为 。

代码如下：

```

1 // 求 x 的汉明权重
2 int popcount(int x) {
3     int cnt = 0;
4     while (x) {
5         cnt++;
6         x -= x & -x;
7     }
8     return cnt;
9 }

```

构造汉明权重递增的排列

在 [状压 DP](#) 中，按照 `popcount` 递增的顺序枚举有时可以避免重复枚举状态。这是构造汉明权重递增的排列的一大作用。

下面我们来具体探究如何在 时间内构造汉明权重递增的排列。

我们知道，一个汉明权重为 的最小的整数为 。只要可以在常数时间构造出一个整数汉明权重相等的后继，我们就可以通过枚举汉明权重，从 开始不断寻找下一个数的方式，在 时间内构造出 的符合要求的排列。

而找出一个数 汉明权重相等的后继有这样的思路，以 为例：

- 把 最右边的 向左移动，如果不能移动，移动它左边的 ，以此类推，得到 。
- 把得到的 最后移动的 原先的位置一直到最低位的所有 都移到最右边。这里最后移动的 原来在第三位，所以最后三位 要变成 ，得到 。

这个过程可以用位运算优化：

```
1 int t = x + (x & -x);
2 x = t | (((t&-t)/(x&-x))>>1)-1);
```

- 第一个步骤中，我们把数 加上它的 lowbit，在二进制表示下，就相当于把 最右边的连续一段 换成它左边的一个。如刚才提到的二进制数，它在加上它的 lowbit 后是。这其实得到了我们答案的前半部分。
- 我们接下来要把答案后面的 补齐，的 lowbit 是 最右边连续一段 最左边的 移动后的位置，而 的 lowbit 则是最右边连续一段 最左边的位置。还是以 为例，，，。
- 接下来的除法操作是这种位运算中最难理解的部分，但也是最关键的部分。我们设原数最右边连续一段 最高位的 在第 位上（位数从 开始），最低位的 在第 位，的 lowbit 等于 $1 \ll (r+1)$ ，的 lowbit 等于 $1 \ll l$ ， $((t\&-t)/(x\&-x))\gg 1$ 得到的，就是 $(1\ll(r+1))/(1\ll l)/2 = (1\ll(r-l))/2 = 1\ll(r-l)$ ，在二进制表示下就是 后面跟上 个零，零的个数正好等于连续 的个数减去。举我们刚才的数为例，。把这个数减去 得到的就是我们要补全的低位，或上原来的数就可以得到答案。

所以枚举 按汉明权重递增的排列的完整代码如下：

```
1 for (int i = 0; (1<<i)-1 <= n; i++) {
2     for (int x = (1<<i)-1, t; x <= n; t = x+(x&-x), x = x ? (t|(((t&-t)/(x&-x))>>1)-1) : (n+1)) {
3         // 写下需要完成的操作
4     }
5 }
```

其中要注意 的特判，因为 没有相同汉明权重的后继。

内建函数

GCC 中还有一些用于位运算的内建函数：

1. `int __builtin_ffs(int x)`：返回 的二进制末尾最后一个 的位置，位置的编号从 开始（最低位编号为 ）。当 为 时返回 。
2. `int __builtin_clz(unsigned int x)`：返回 的二进制的前导 的个数。当 为 时，结果未定义。
3. `int __builtin_ctz(unsigned int x)`：返回 的二进制末尾连续 的个数。当 为 时，结果未定义。
4. `int __builtin_clrsb(int x)`：当 的符号位为 时返回 的二进制的前导 的个数减一，否则返回 的二进制的前导 的个数减一。
5. `int __builtin_popcount(unsigned int x)`：返回 的二进制中 的个数。
6. `int __builtin_parity(unsigned int x)`：判断 的二进制中 的个数的奇偶性。

这些函数都可以在函数名末尾添加 `l` 或 `ll`（如 `__builtin_popcountll`）来使参数类型变为 `(unsigned)long` 或 `(unsigned)long long`（返回值仍然是 `int` 类型）。例如，我们有时候希望求出一个数以二为底的对数，如果不考虑 0 的特殊情况，就相当于这个数二进制的位数 -1，而一个数 `n` 的二进制表示的位数可以使用 `32-__builtin_clz(n)` 表示，因此 `31-__builtin_clz(n)` 就可以求出 `n` 以二为底的对数。

由于这些函数是内建函数，经过了编译器的高度优化，运行速度十分快（有些甚至只需要一条指令）。

更多位数

如果需要操作的集合非常大，可以使用 [bitset](#)。

题目推荐

参考资料与注释

1. 位运算技巧: <https://graphics.stanford.edu/~seander/bithacks.html>
2. Other Builtins of GCC: <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html>

-
1. 适用于 C++14 以前的标准。在 C++14 和 C++17 标准中，若原值为带符号类型，且移位后的结果能被原类型的无符号版本容纳，则将该结果 [转换](#) 为相应的带符号值，否则行为未定义。在 C++20 标准中，规定了无论是带符号数还是无符号数，左移均直接舍弃移出结果类型的位。 [↩](#)
 2. 适用于 C++20 以前的标准。 [↩](#)
 3. 这种右移方式称为算术右移。在 C++20 以前的标准中，并没有规定带符号数右移运算的实现方式，大多数平台均采用算术右移。在 C++20 标准中，规定了带符号数右移运算是算术右移。 [↩](#)
 4. 一个数二进制表示从低往高的第一个 连同后面的零，如 的 `lowbit` 是，详见 [树状数组](#)。 [↩](#)
-

本页面最近更新: 2023/11/21 16:12:29, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Marcythm](#), [ouuan](#), [StudyingFather](#), [akakw1](#), [Anguei](#), [aofall](#), [billchenchina](#), [CCXXXI](#), [cjsoft](#), [Dian-Jiao](#), [diauweb](#), [Early0v0](#), [Enter-tainer](#), [flylai](#), [Great-designer](#), [greyqz](#), [H-J-Granger](#), [Henry-ZHR](#), [hjsjhn](#), [iamtwz](#), [Ir1d](#), [Konano](#), [ksyx](#), [lihaoyu1234](#), [Link-cute](#), [Menci](#), [MingqiHuang](#), [orzAtalod](#), [Planarialce](#), [sakuragi1111](#), [sbofgayschool](#), [shawlleyw](#), [Shen-Linwood](#), [skippre](#), [sshwy](#), [stevenlele](#), [Tiphereth-A](#), [Voileexperiments](#), [xinchengo](#), [ylxmf2005](#), [zryi2003](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用